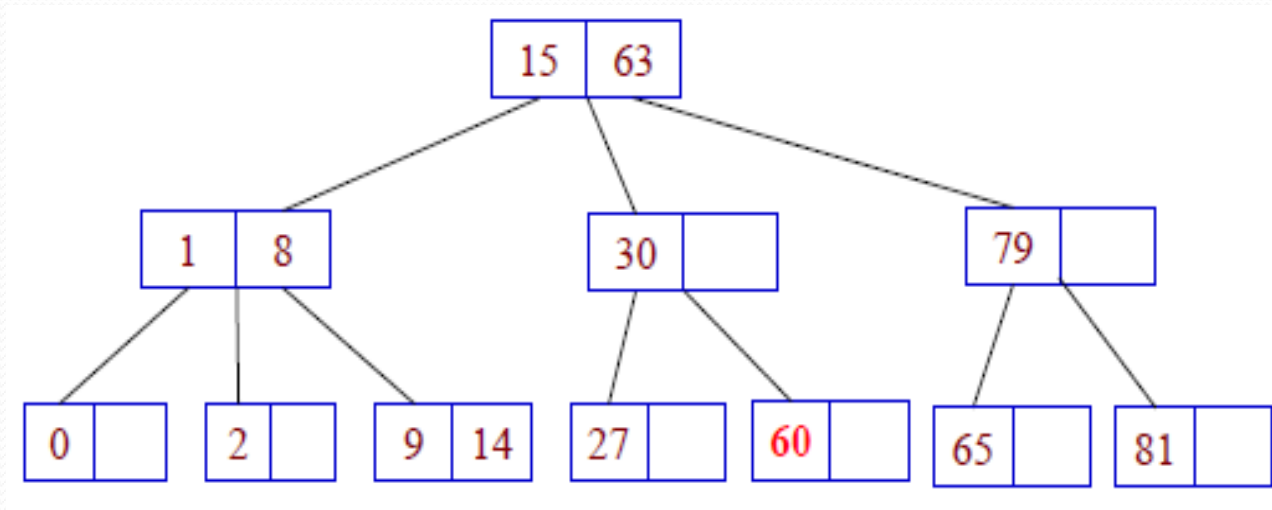


Árbol 2-3

Algoritmos y Estructura de Datos II

Árboles 2 -3. Concepto

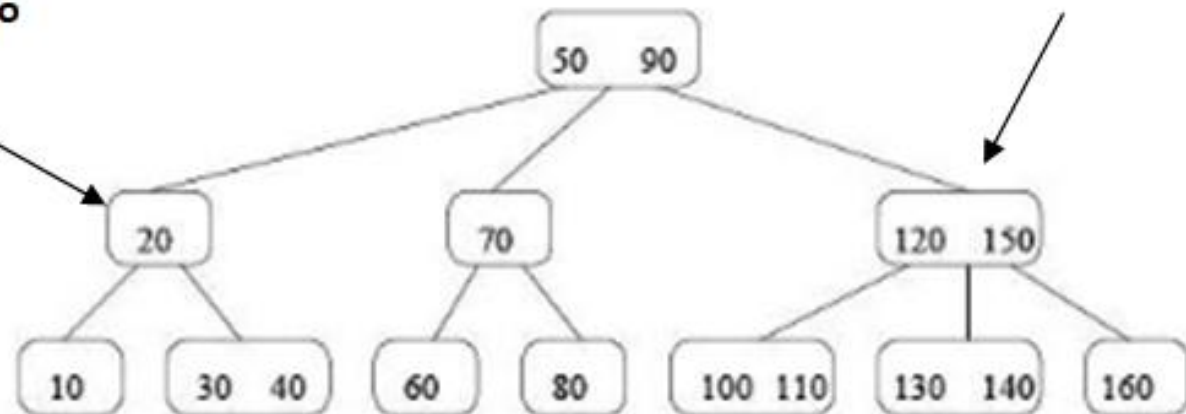
Son árboles cuyos nodos pueden contener hasta dos hijos, por lo tanto un nodo interno puede tener 2 o 3 hijos, dependiendo de cuantos elementos posea el nodo.



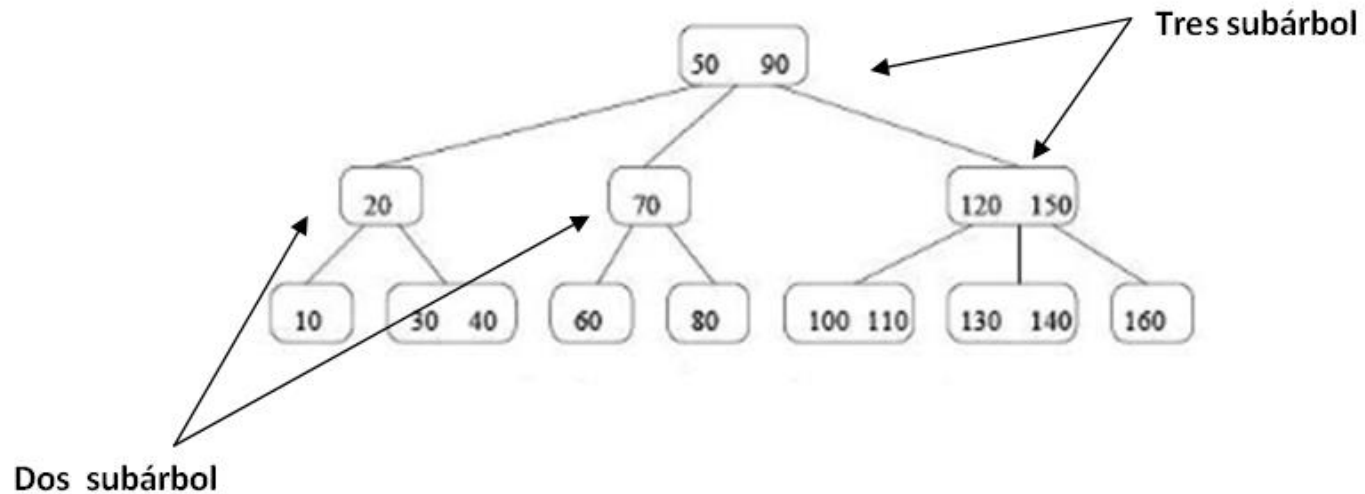
Cada nodo tiene 1 o 2 elementos

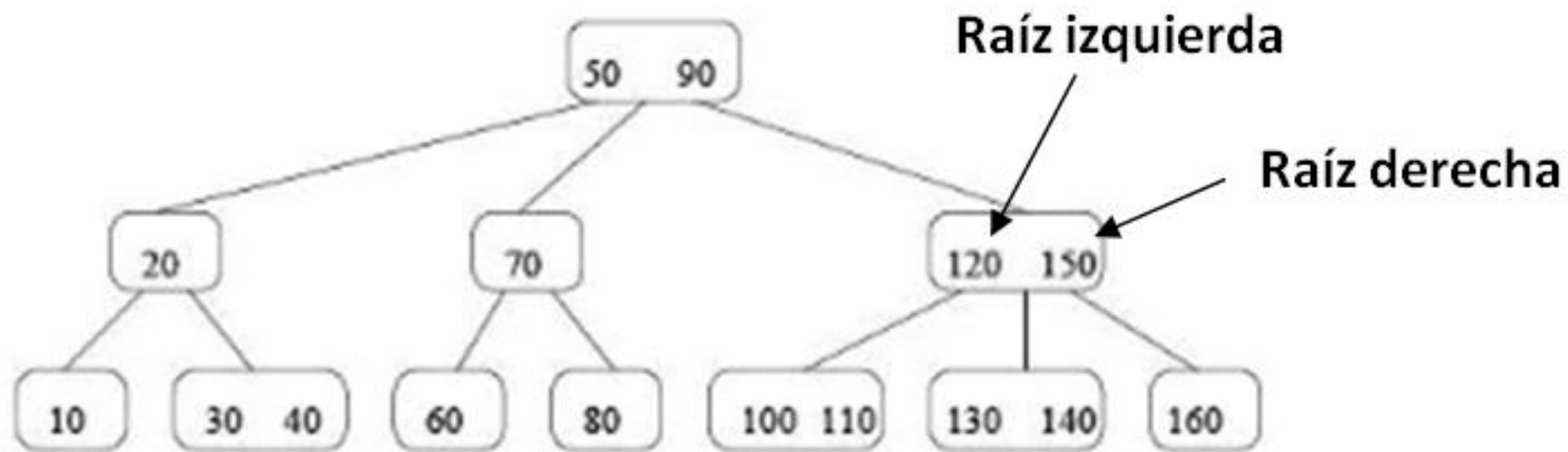
Nodo con un elemento

Nodo con dos elementos



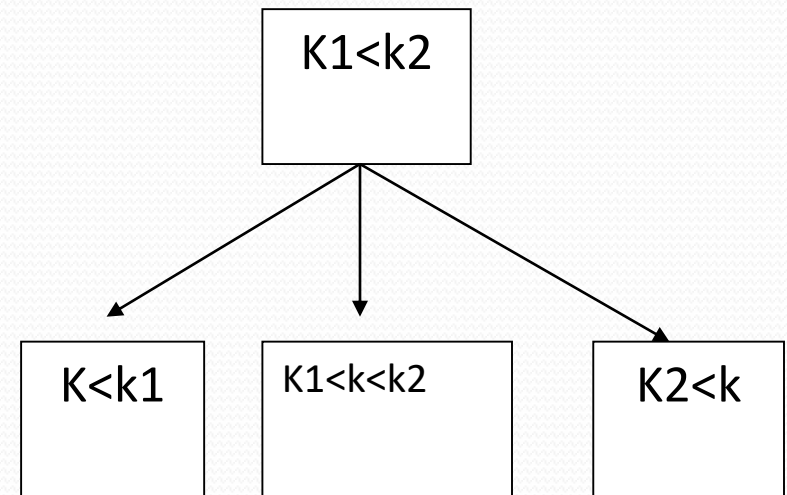
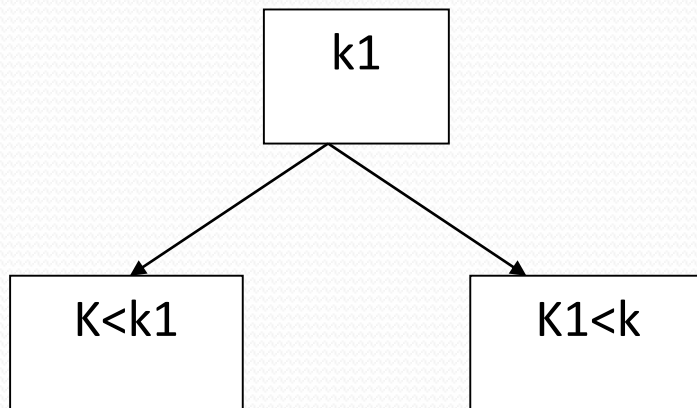
- Cada nodo tiene subárbol asociado





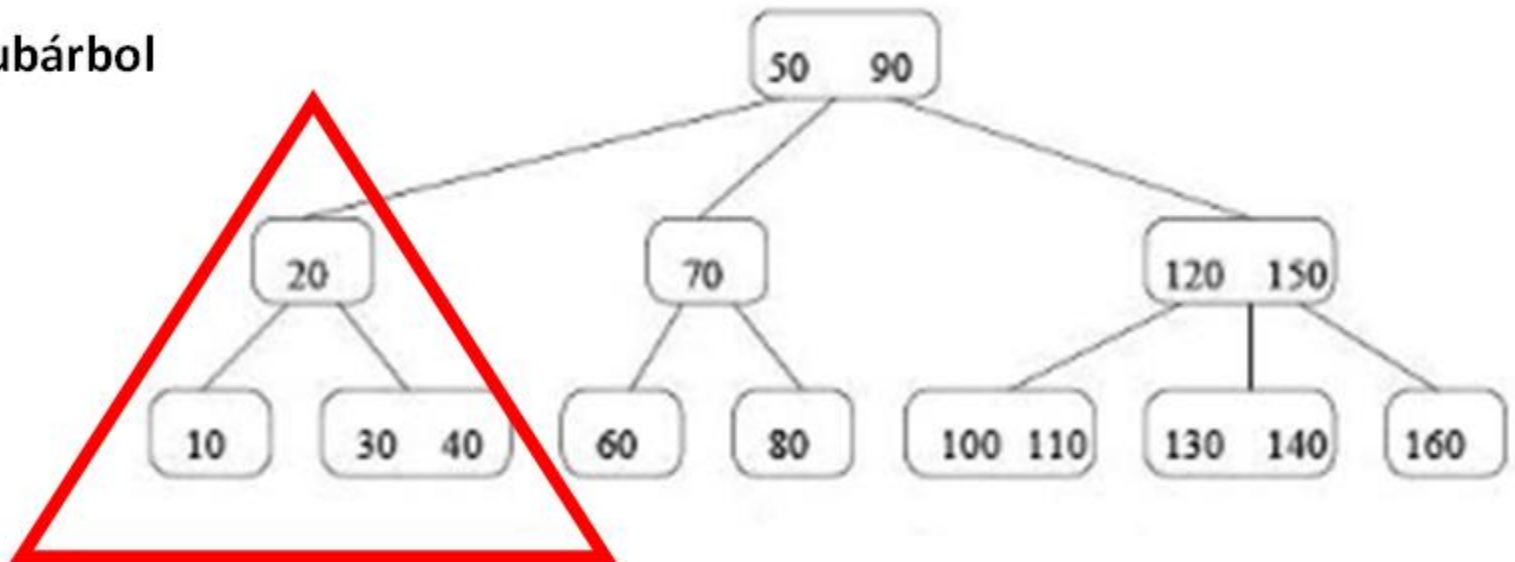
Árbol 2-3 Ordenado

- En el árbol 2-3 no hay elementos repetidos
- Son un tipo de árbol balanceado por altura
- El elemento de la izquierda de cada nodo es menor que el elemento de su derecha.



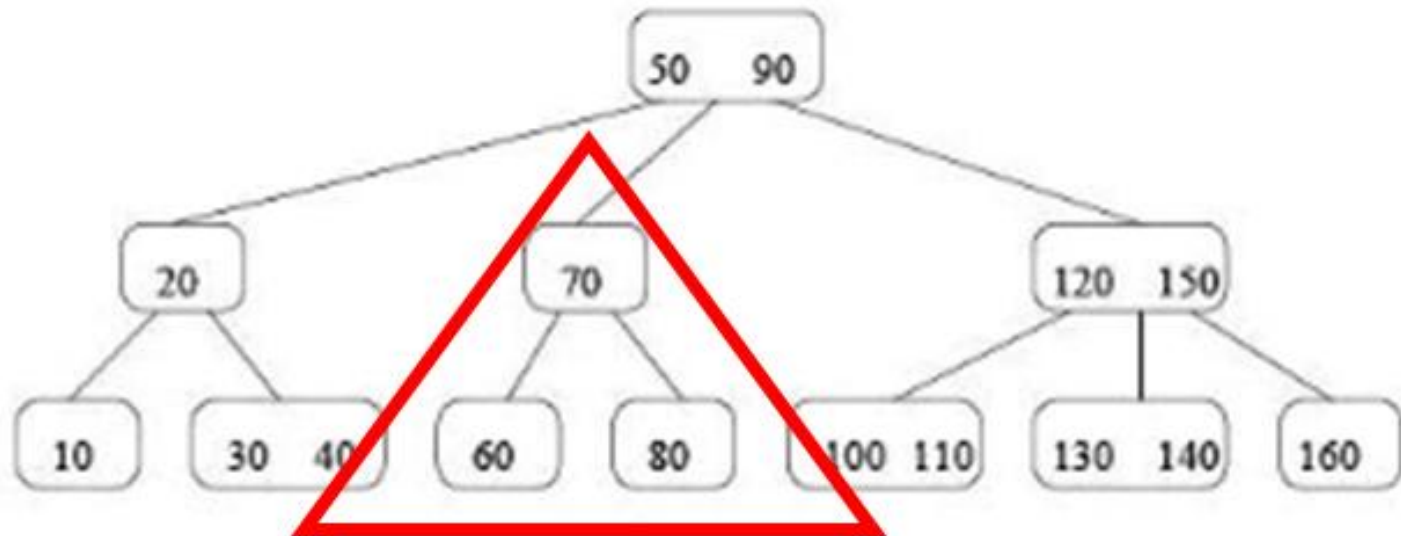
- El primer subárbol es un árbol que contiene elementos menores que la raíz izquierda

Primer subárbol



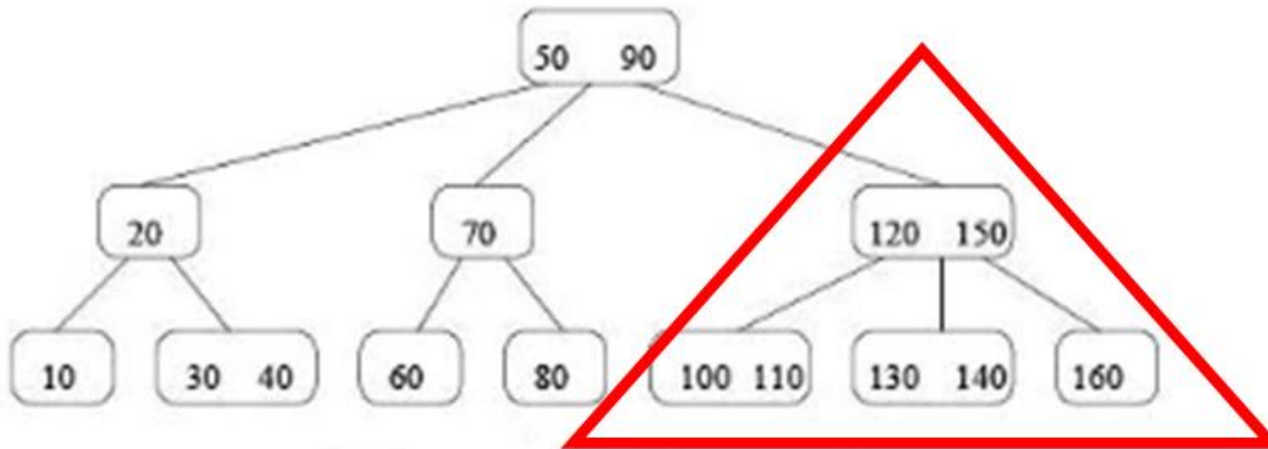
- El segundo subárbol es un árbol que contienen elementos mayores que la raíz izquierda pero menor que la raíz derecha

Segundo subárbol



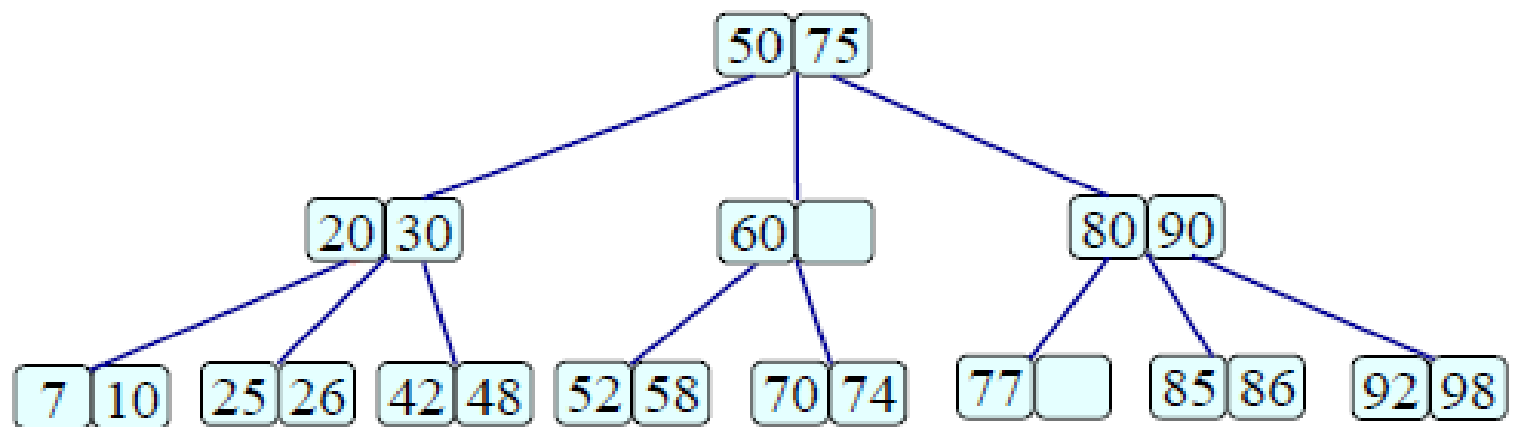
El tercer subárbol es un árbol que contiene los elementos mayores que la raíz derecha.

Tercer subárbol



- 
- El árbol 2-3 es un árbol ordenado balanceado
 - Todas las hojas se encuentran en el mismo nivel ordenadas de izquierda a derecha
 - Todos los nodos internos tienen por lo menos dos subárbol asociados no vacíos, aunque la raíz derecha este vacía

Búsqueda en un árbol 2-3



Búsqueda en un árbol 2-3

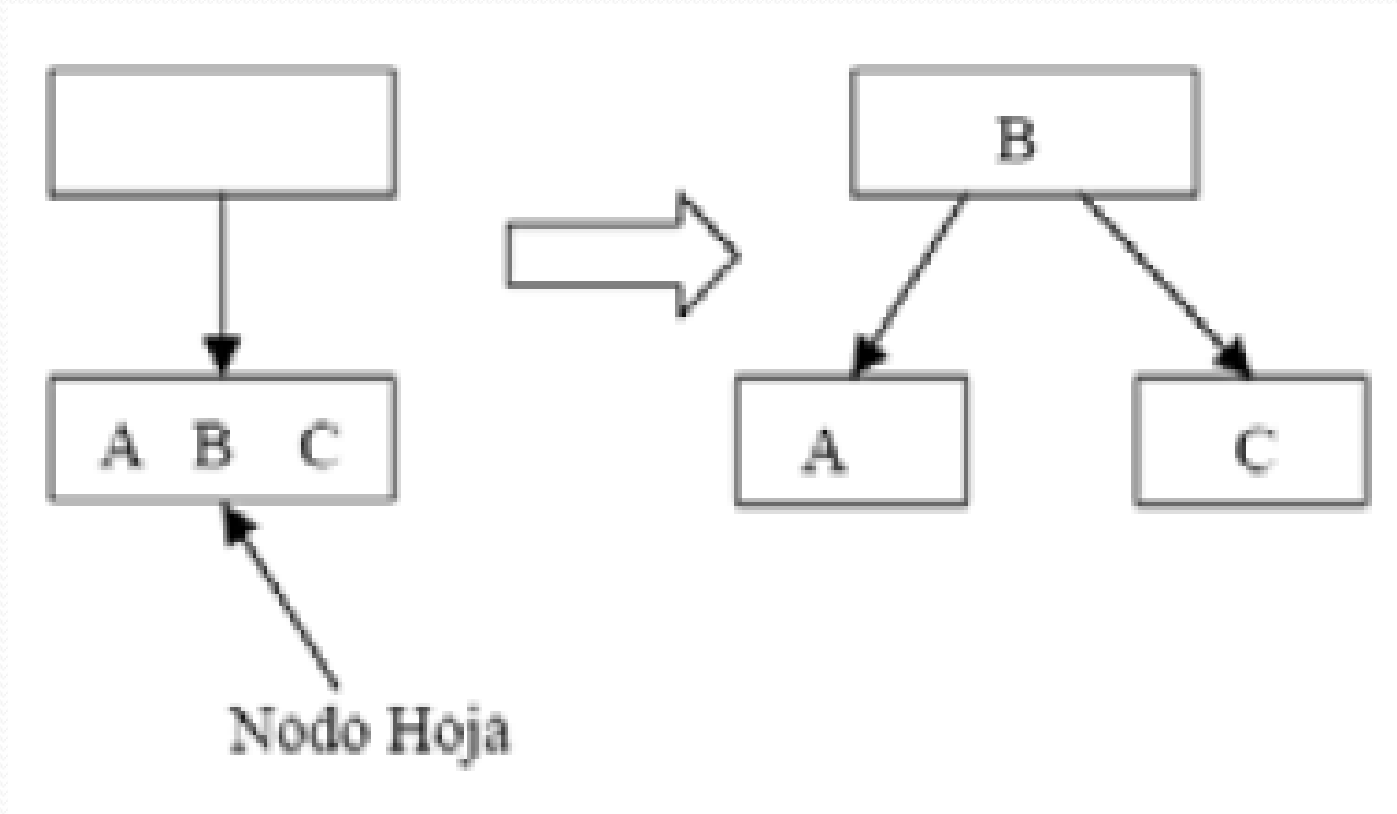
- Se empieza por la raíz y se va bajando por el árbol hasta que se encuentre el dato o se llegue a una hoja.
 - Si el árbol está vacío, el dato no está en el árbol
 - Si no está vacío, primero busca en la raíz
 - Si es alguno de los elementos de la raíz, se encontró.
 - En caso contrario:
 - Si el nodo es una hoja y el elemento no está en el nodo, el algoritmo finaliza,

- Si el nodo no es una hoja:
- Si solo hay un elemento en el nodo:
 - Si es mayor que el dato a buscar, se sigue por el hijo derecho
 - Si es menor se sigue por el hijo izquierdo
- Si en el nodo hay dos elementos x_1 y x_2 (con $x_1 \leq x_2$)
 - Si el dato es menor que x_1 , busca por el hijo izquierdo
 - Si el dato es mayor que x_1 y menor que x_2 , busca por el hijo central
 - Si es mayor que x_2 busca por el hijo derecho

Inserción en el árbol 2-3

Al insertar un nuevo dato en un árbol 2-3 se hará de forma que se mantenga el equilibrio en el árbol.

1. Localizar el nodo hoja donde debe ir el elemento
2. Si hay un solo elemento en ese nodo, añadir el dato
3. Si no hay espacio en el nodo para un nuevo elemento, se divide el nodo y el elemento central se inserta en el nodo padre.



- 
4. Si el nodo padre está lleno, al insertar el nuevo elemento se debe repetir el paso 3.
 5. La secuencia de inserciones y divisiones se puede propagar hacia arriba hasta llegar a la raíz.
 6. Cuando la raíz pase a tener 3 elementos (x_1 , x_2 y x_3) se divide en 2 nuevos nodos x_1 y x_2 , creando una nueva raíz contendrá sólo el elemento x_2
 7. De esta forma, cuando el árbol 2-3 crece en altura, lo hace por arriba, creando una nueva raíz con solo un elemento.

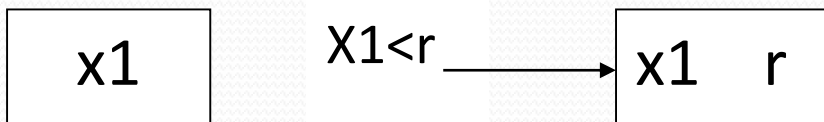


Las inserciones en un árbol 2-3 tienen dos casos:

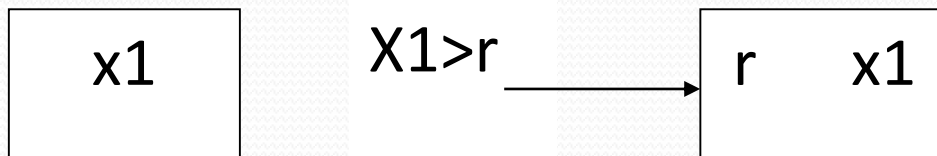
1. Hay espacio en un nodo pues hay un solo elemento
2. No hay espacio y el nodo debe dividirse en dos y la mediana de los tres elementos se inserta en el padre recursivamente.
 - Esto puede generar una secuencia de divisiones hasta la raíz.

En todos los casos, el elemento a insertar se denota por r

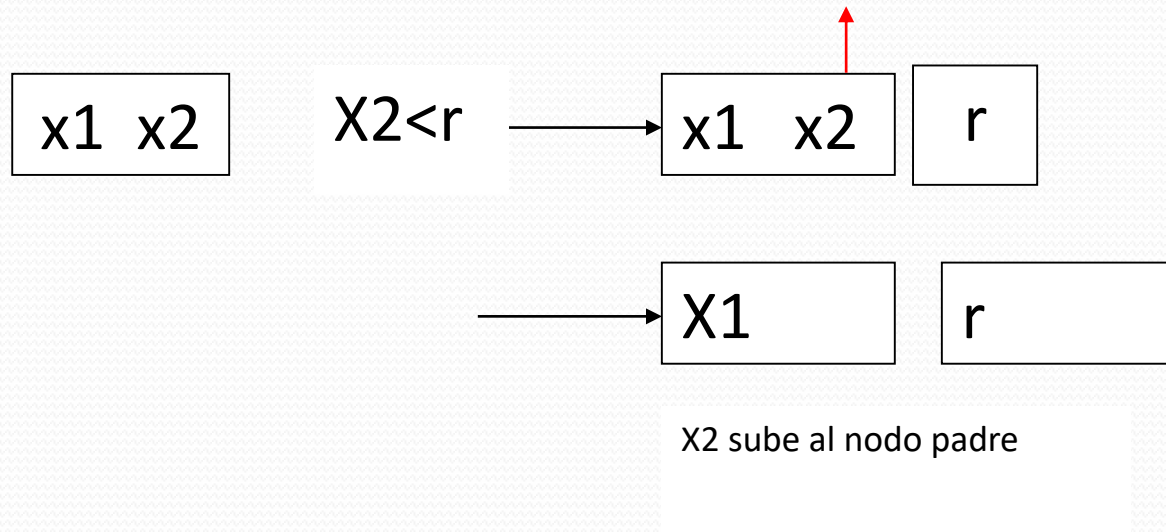
- Caso 1:



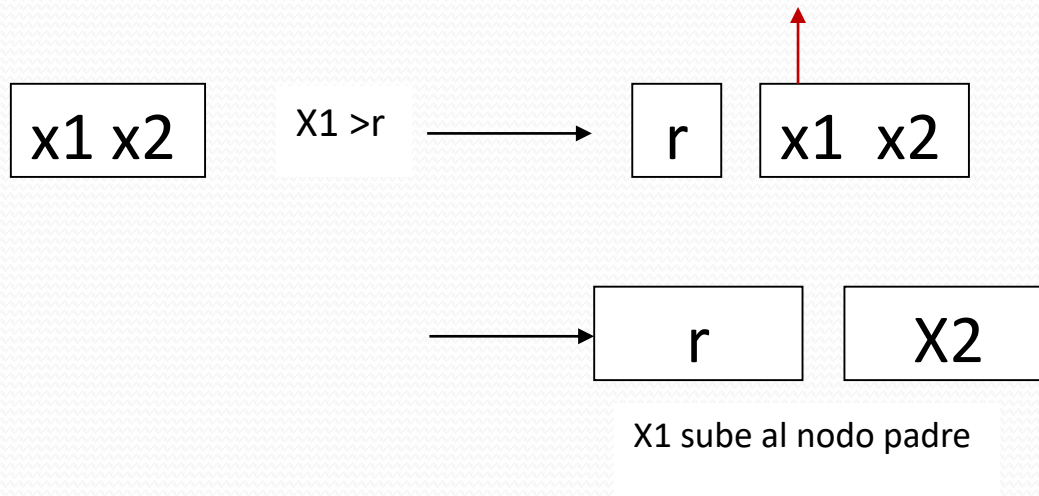
- Caso 2:



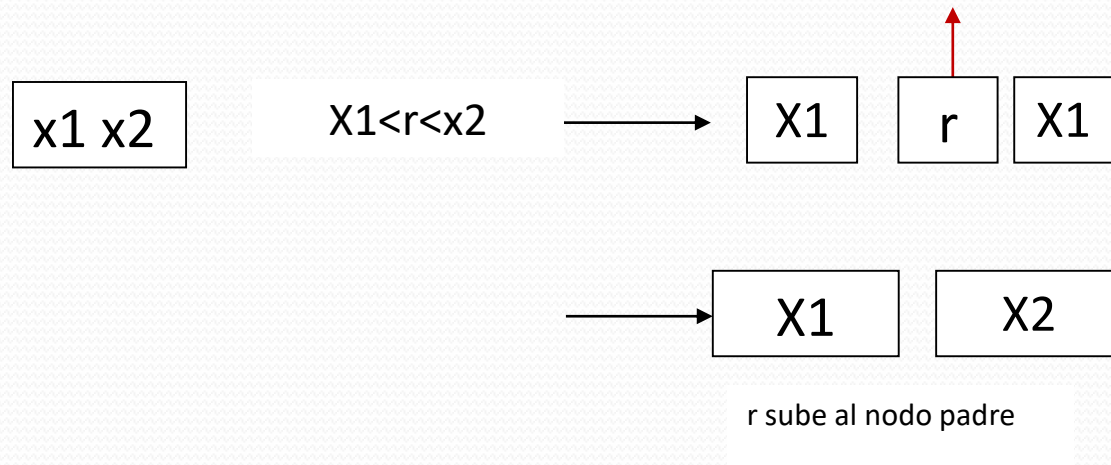
- Caso 3



- Caso 4



- Caso 5



Si el árbol esta vacio

Crea un nuevo nodo y coloca x_1 en el lado izquierdo del nodo

Sino

Si hay un elemento y existe espacio en el nodo.

Si x_1 es menor que el elemento, el elemento se coloca a la derecha

Sino

si x_1 es mayor que el elemento

El elemento se coloca al lado izquierda y x_1 al lado derecho

sino Si el nodo está lleno

Se divide en dos nodos del mismo nivel

Se crea un nuevo nodo y se reparten los tres elementos.

Después

Si elementos es mayor que x_2 , sube x_2 a su padre

Si elemento es menor que x_1 , sube x_1 , a su padre

Si el elemento es mayor que x_1 , pero menor que x_2 , sube el elemento a su padre



EJEMPLO

Ejemplo

Insertar en un árbol2-3 los siguientes elementos:

$S = \{ 30, 2, 15, 63, 65, 1, 0, 14, 27, 8, 9, 81, 79, 60 \}$

Input: 30

30	
----	--

.....

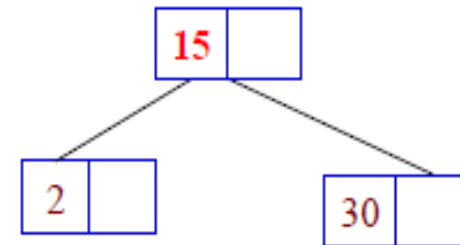
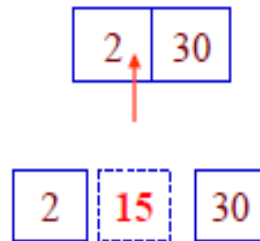
Input: 2

2	30
---	----

Insertar en un árbol2-3 los siguientes elementos:

$S = \{ 30, 2, 15, 63, 65, 1, 0, 14, 27, 8, 9, 81, 79, 60 \}$

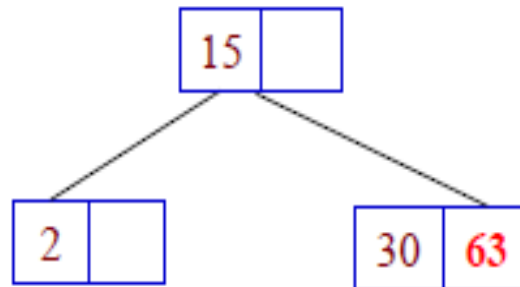
Input: 15



Observe que el árbol crece hacia arriba, por la raíz.

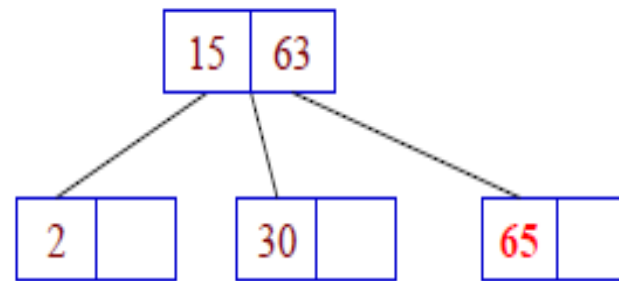
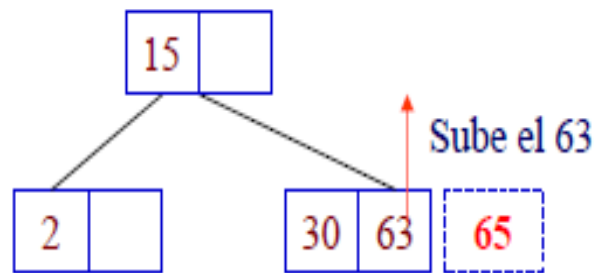
$S = \{ 30, 2, 15, 63, 65, 1, 0, 14, 27, 8, 9, 81, 79, 60 \}$

Input: 63



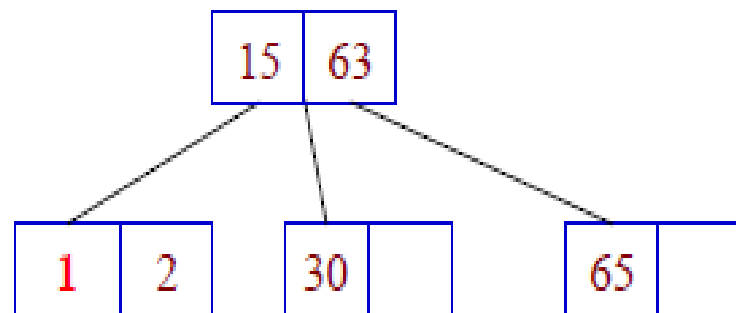
$S = \{ 30, 2, 15, 63, 65, 1, 0, 14, 27, 8, 9, 81, 79, 60 \}$

Input: 65



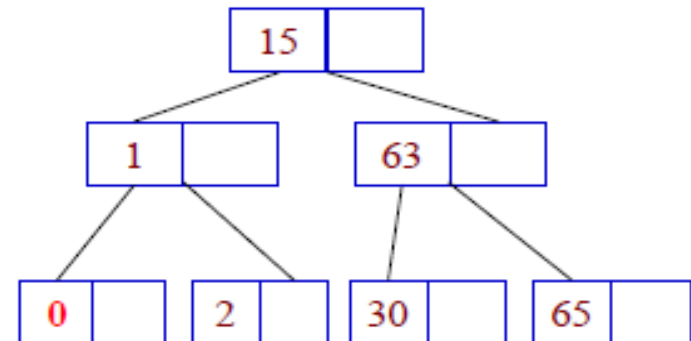
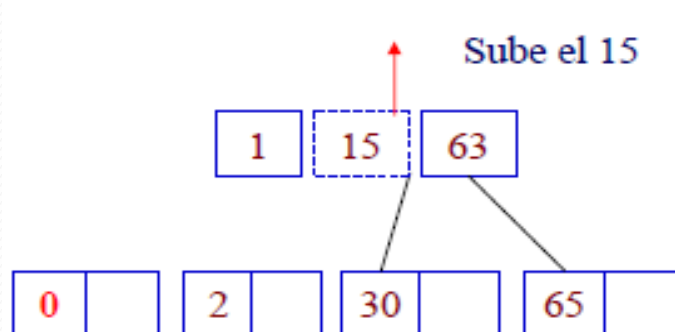
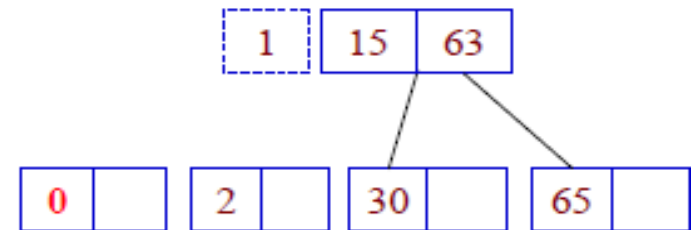
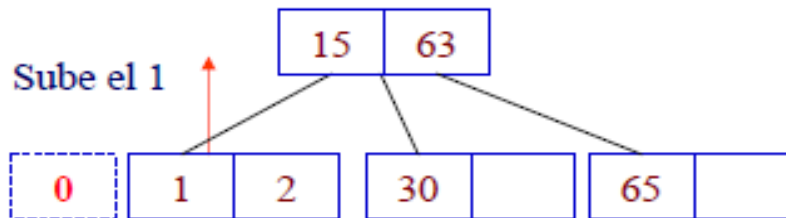
$S = \{ 30, 2, 15, 63, 65, 1, 0, 14, 27, 8, 9, 81, 79, 60 \}$

Input: 1



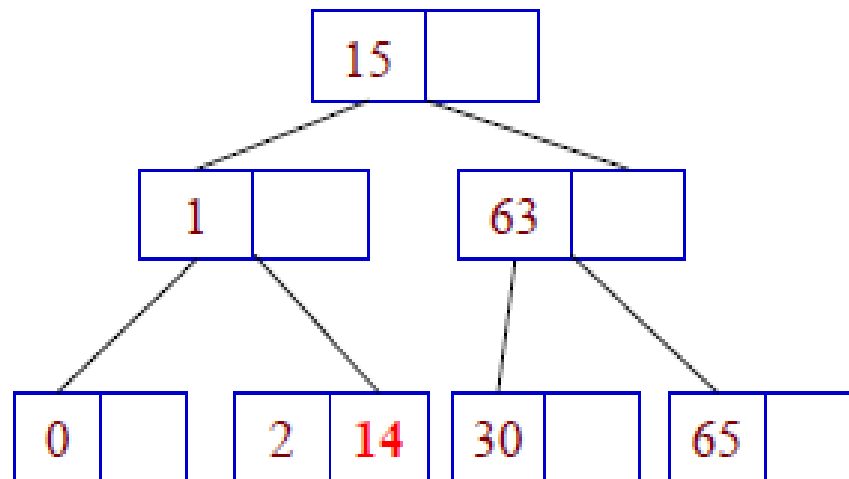
$S = \{ 30, 2, 15, 63, 65, 1, 0, 14, 27, 8, 9, 81, 79, 60 \}$

Input: 0



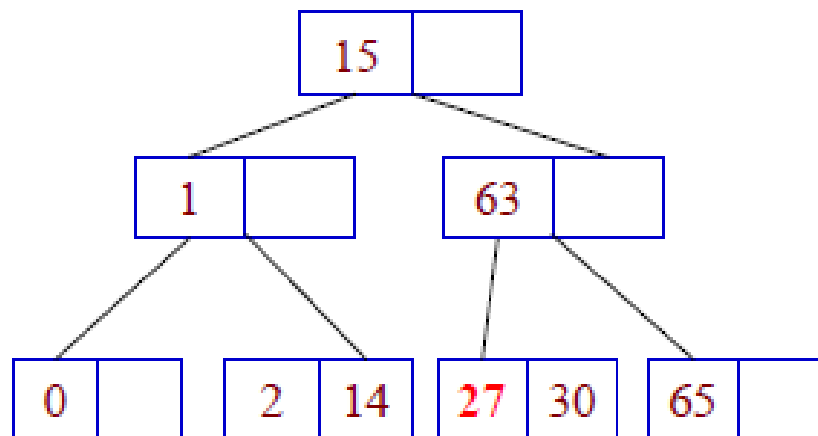
$S = \{ 30, 2, 15, 63, 65, 1, 0, 14, 27, 8, 9, 81, 79, 60 \}$

Input: 14



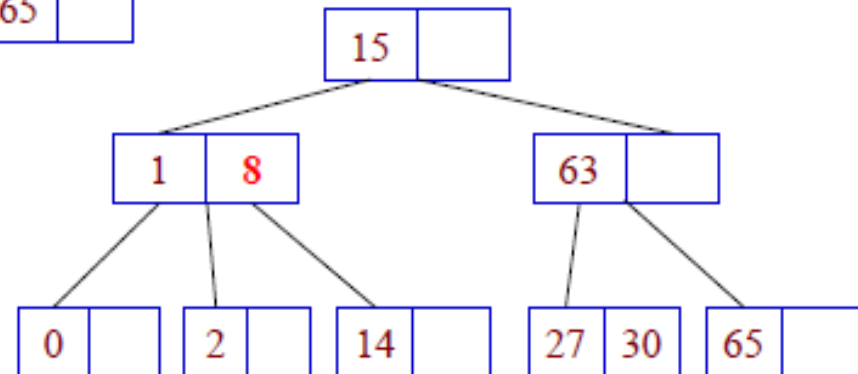
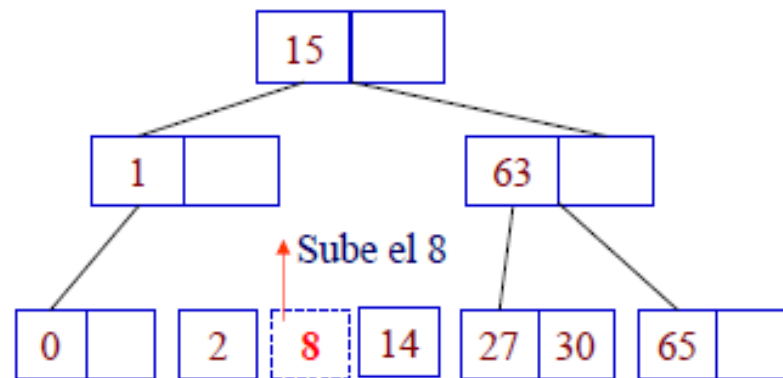
$S = \{ 30, 2, 15, 63, 65, 1, 0, 14, 27, 8, 9, 81, 79, 60 \}$

Input: 27



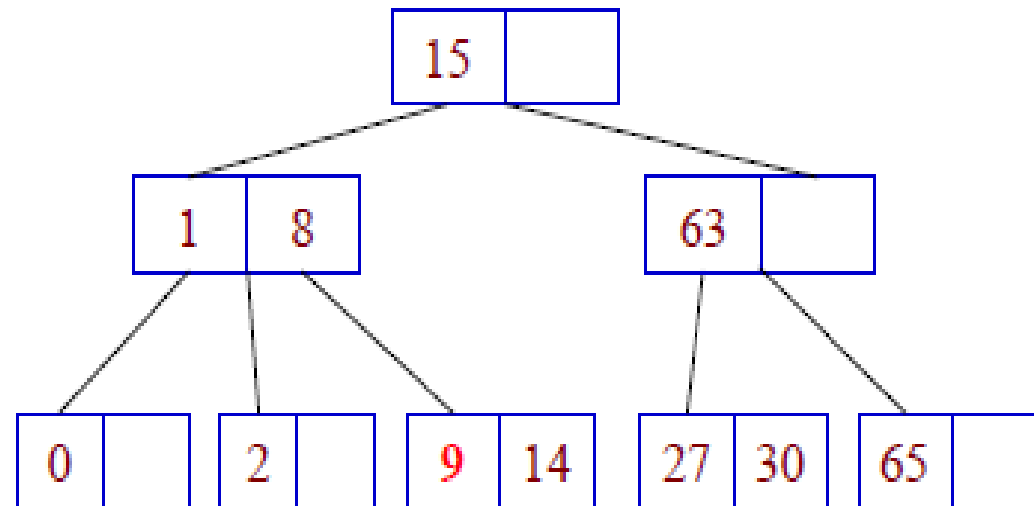
$S = \{ 30, 2, 15, 63, 65, 1, 0, 14, 27, 8, 9, 81, 79, 60 \}$

Input: 8



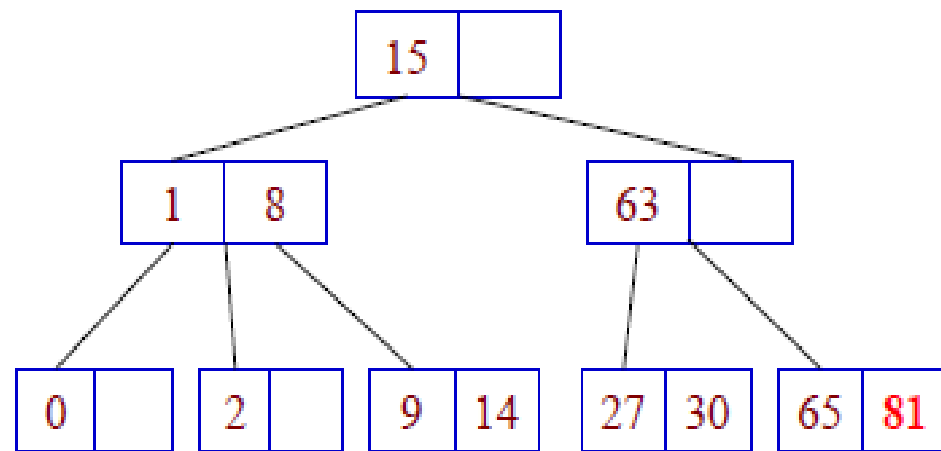
$S = \{ 30, 2, 15, 63, 65, 1, 0, 14, 27, 8, 9, 81, 79, 60 \}$

Input: 9



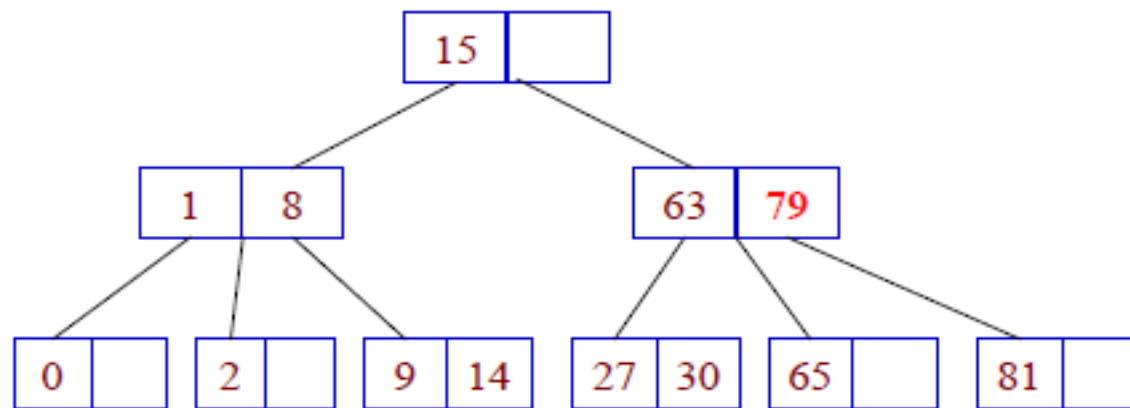
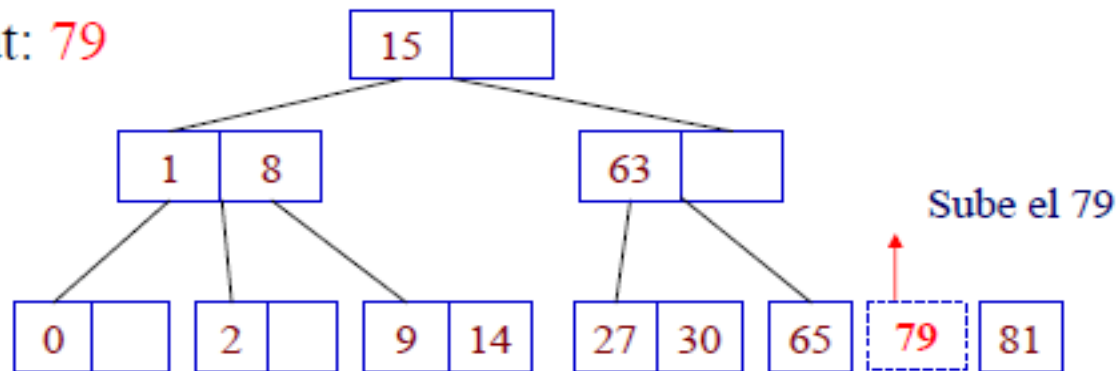
$S = \{ 30, 2, 15, 63, 65, 1, 0, 14, 27, 8, 9, 81, 79, 60 \}$

Input: 81



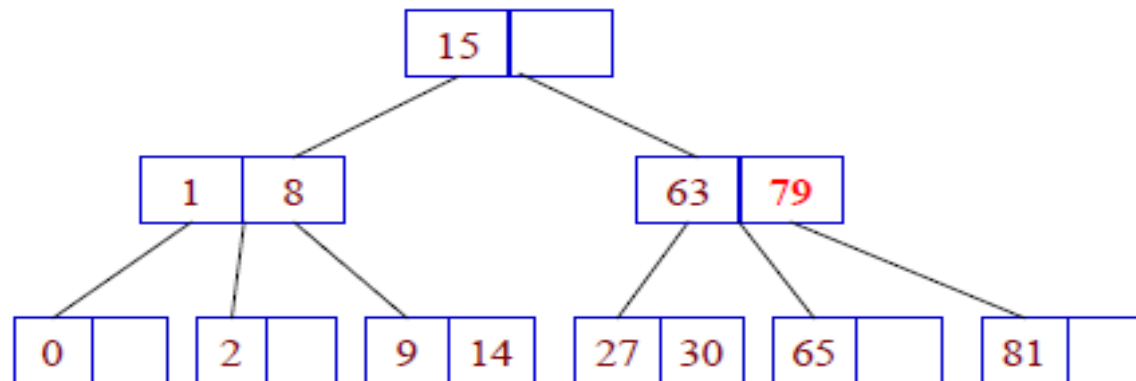
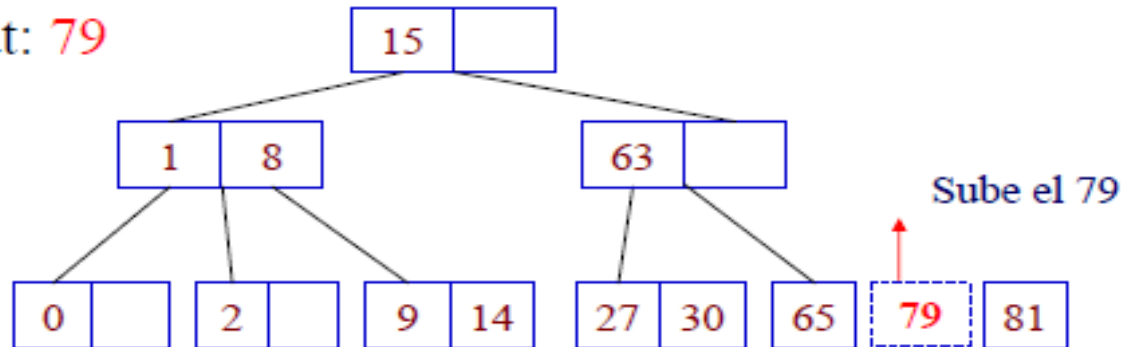
$S = \{ 30, 2, 15, 63, 65, 1, 0, 14, 27, 8, 9, 81, 79, 60 \}$

Input: 79



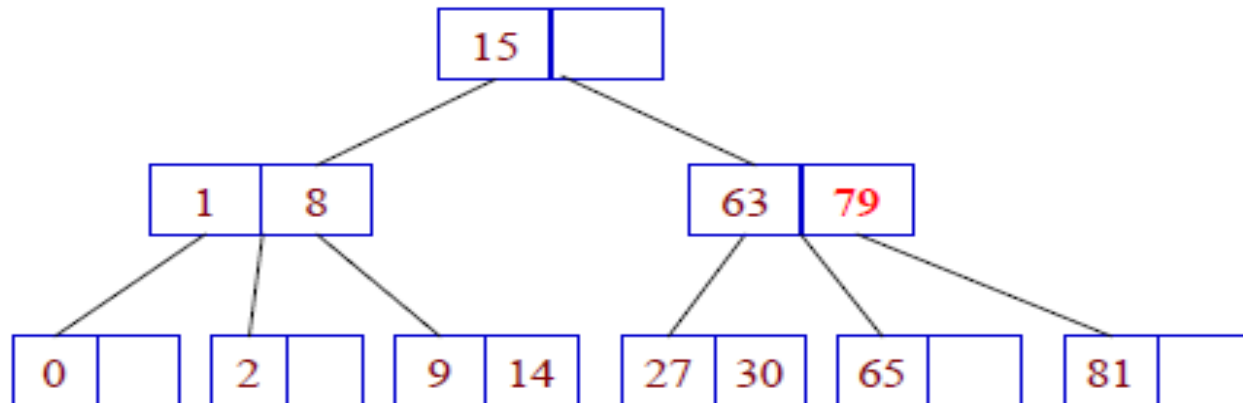
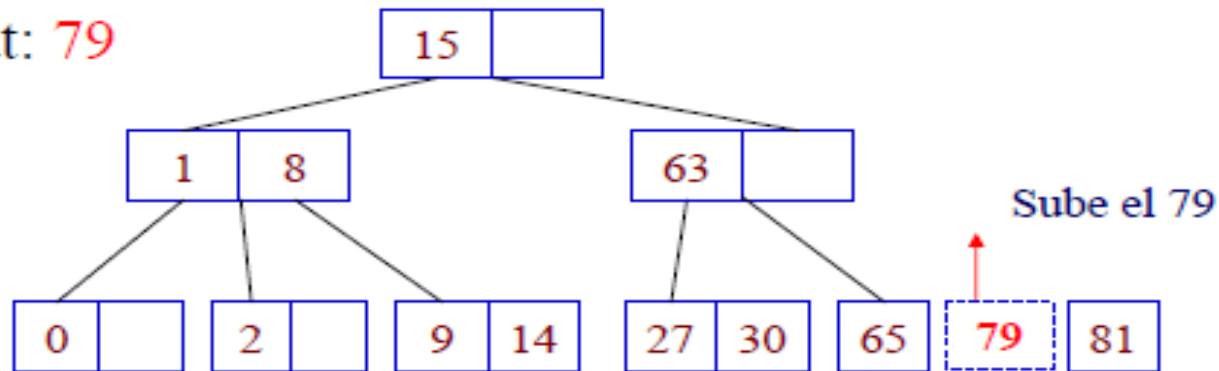
$S = \{ 30, 2, 15, 63, 65, 1, 0, 14, 27, 8, 9, 81, 79, 60 \}$

Input: 79



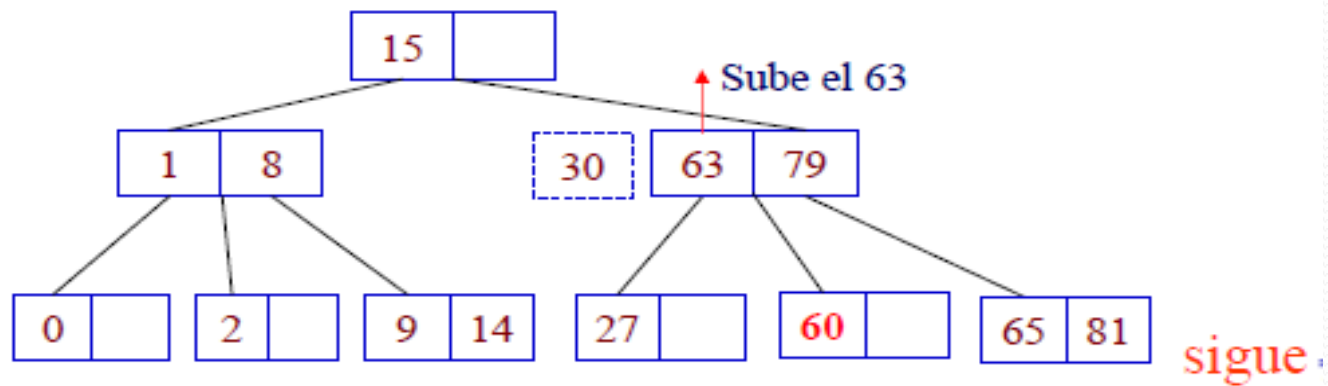
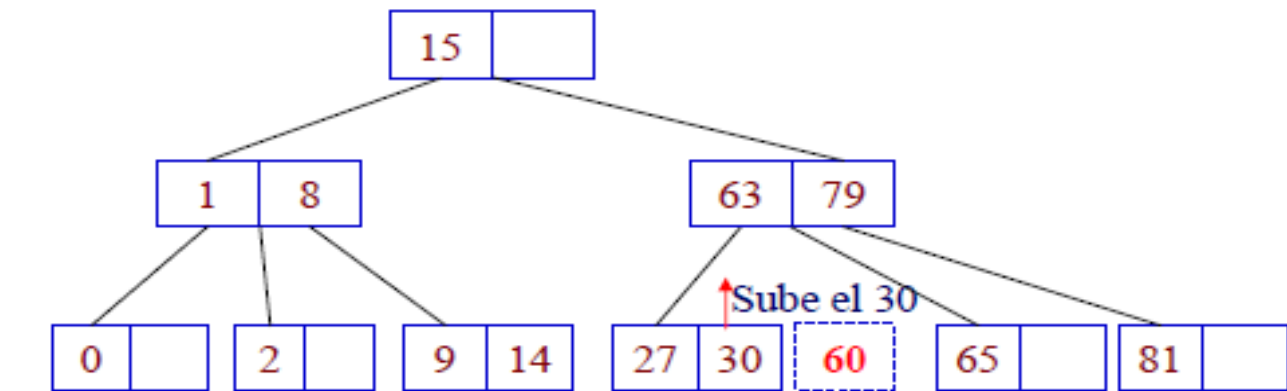
$S = \{ 30, 2, 15, 63, 65, 1, 0, 14, 27, 8, 9, 81, 79, 60 \}$

Input: 79



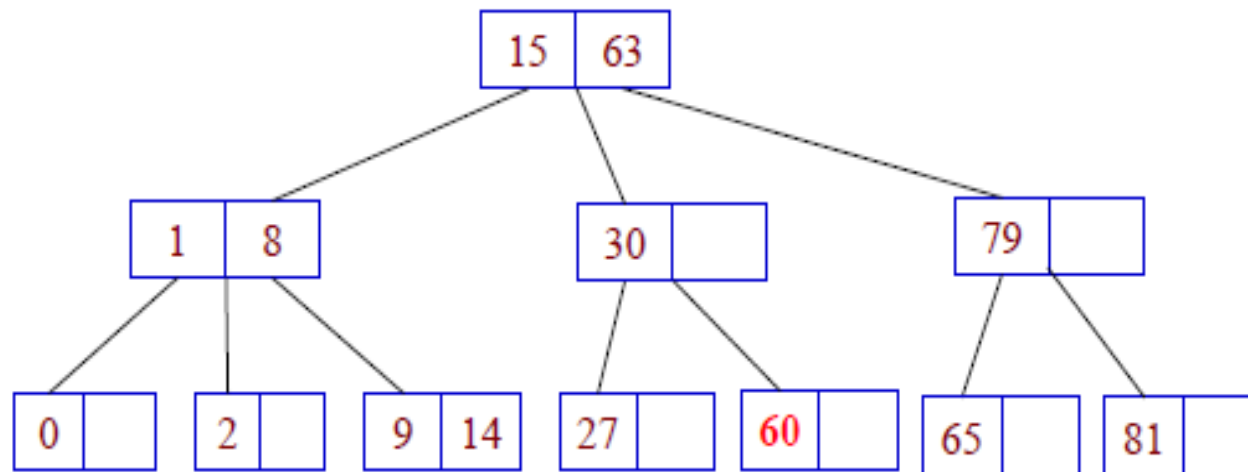
$S = \{ 30, 2, 15, 63, 65, 1, 0, 14, 27, 8, 9, 81, 79, 60 \}$

Input: 60



$S = \{ 30, 2, 15, 63, 65, 1, 0, 14, 27, 8, 9, 81, 79, 60 \}$

Input: 60



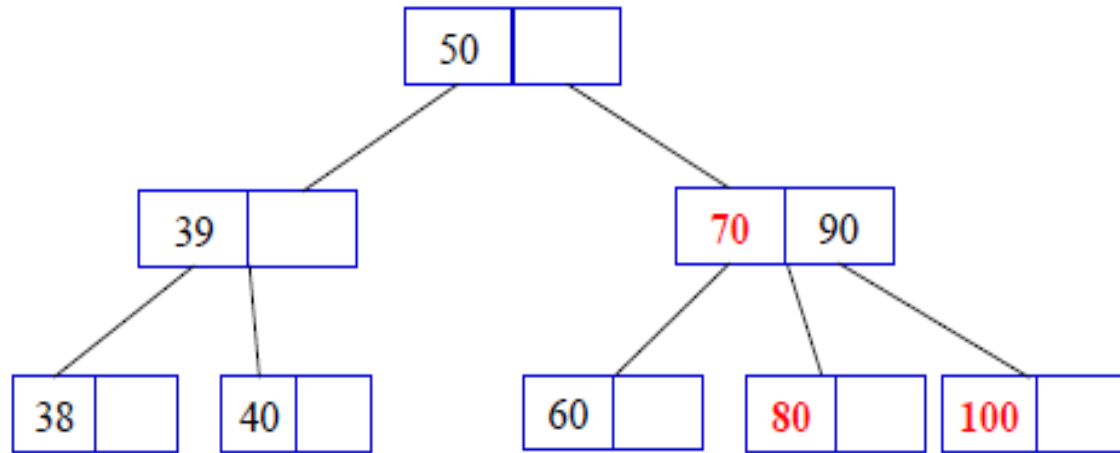
Eliminación un árbol 2-3

- La estrategia de eliminación de datos en un árbol 2-3 es complementación a la inserción.
- En la inserción se produce una cadena de divisiones e inserciones hasta que un nodo no necesite dividirse o se llegue a la raíz.
- En el caso de la eliminación, los nodos se quedan vacíos y se fusionan con sus hermanos, hasta que un nodo quede vacío o se llegue a la raíz.

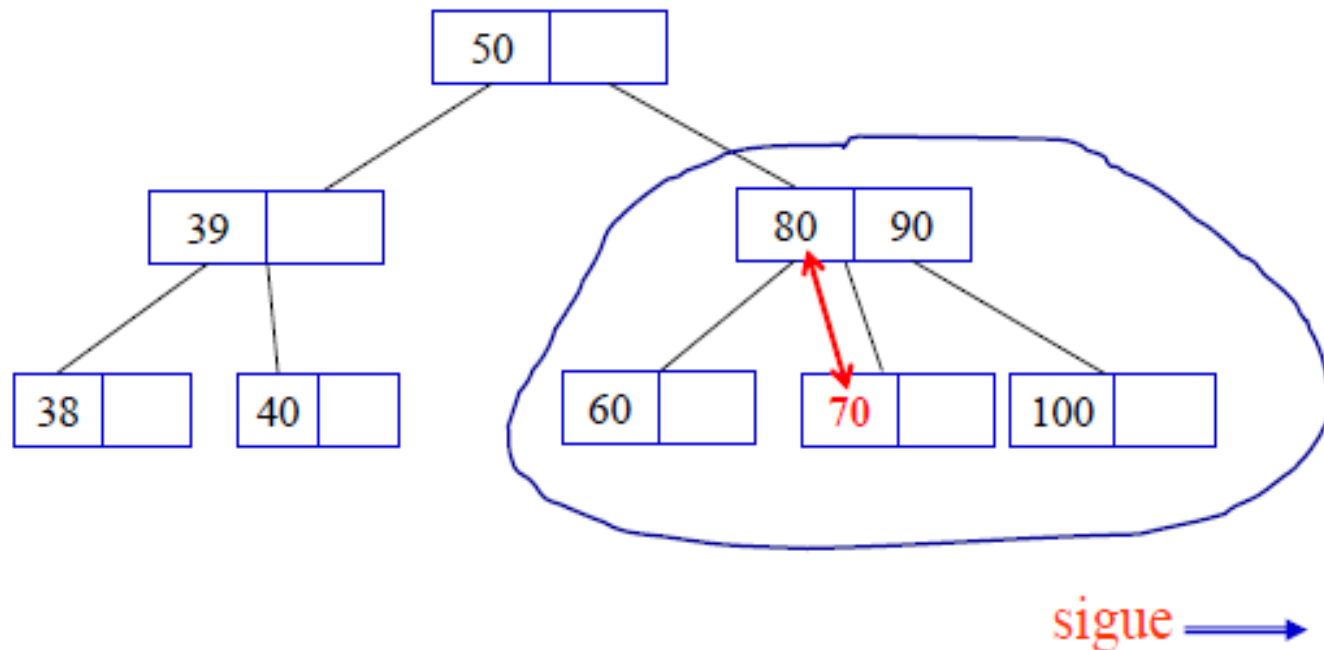
1. El proceso siempre va a empezar en el nodo hoja
 - Si el elemento a borrar está en un nodo interior, se intercambia su valor con el sucesor con el inorden
 - Menos elementos del subárbol que queda a la derecha del elemento a borrar
2. Si en la hoja en que se inicia la eliminación hay otro elemento se elimina
3. Si en la hoja es el único elemento, el nodo queda vacío, lo cual está permitido en árbol 2-3
 - Para arreglarlo, se fusiona el nodo vacío con uno de sus hermanos
 - Como el nodo padre ha perdido un hijo, también tiene que tener un elemento menos, por lo que pasa un elemento del padre al nuevo nodo
4. Si el nodo hermano ya estaba lleno, se divide el nuevo nodo en 2 hijos y el elemento del medio queda como padre.
5. Si el nodo padre se queda vacío al perder un elemento, se debe de repetir el algoritmo de fusión en el árbol tantos niveles hacia arriba como haga falta hasta encontrar un nodo que no se quede vacío o se llegue a la raíz.
 - Si se llega a la raíz y se queda vacía, entonces tiene un hijo
 - La raíz se elimina y su único hijo pasa a ser la nueva raíz

Ejemplo

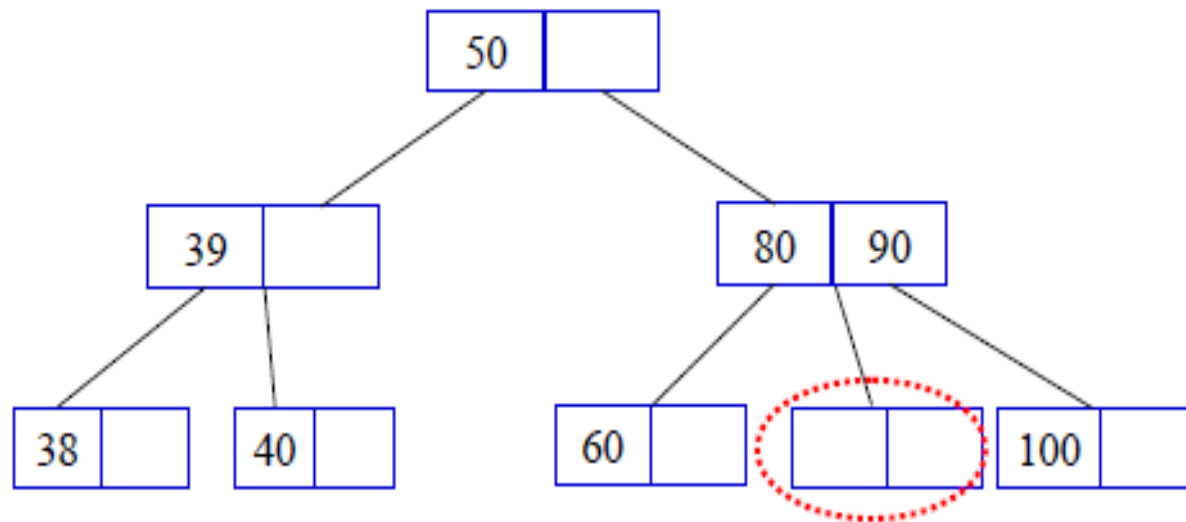
Eliminar del siguiente árbol 2-3 los elementos : 70, 100 y 80.



- Como el elemento **70** no está en una hoja, se intercambia con su sucesor en inorden, el **80**.
- El nodo quedaría entonces [80, 90] con tres hijos, [60], [70] y [100].

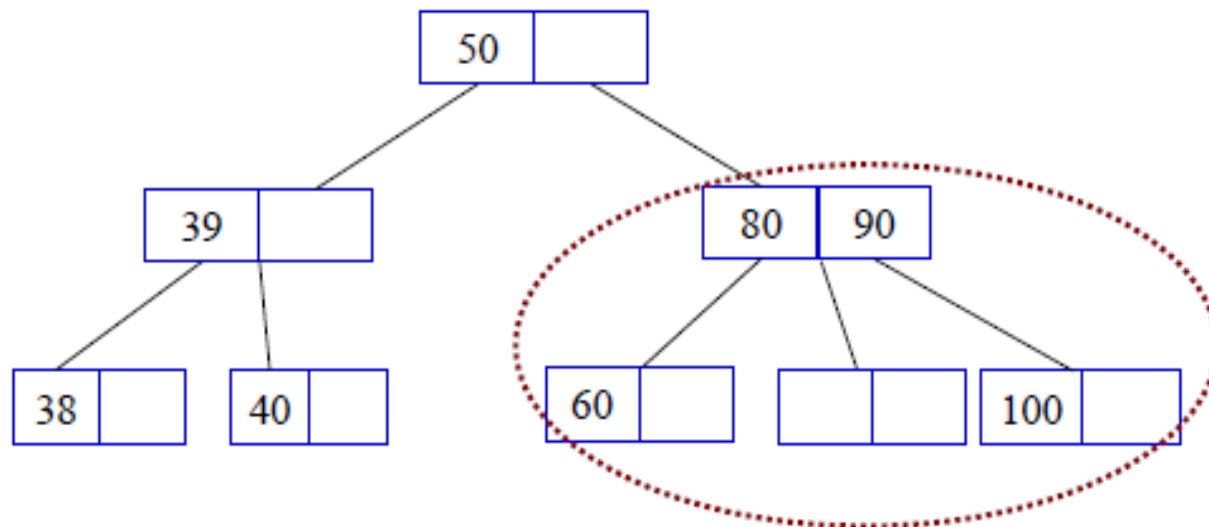


- Al borrar el **70**, el nodo queda vacío.



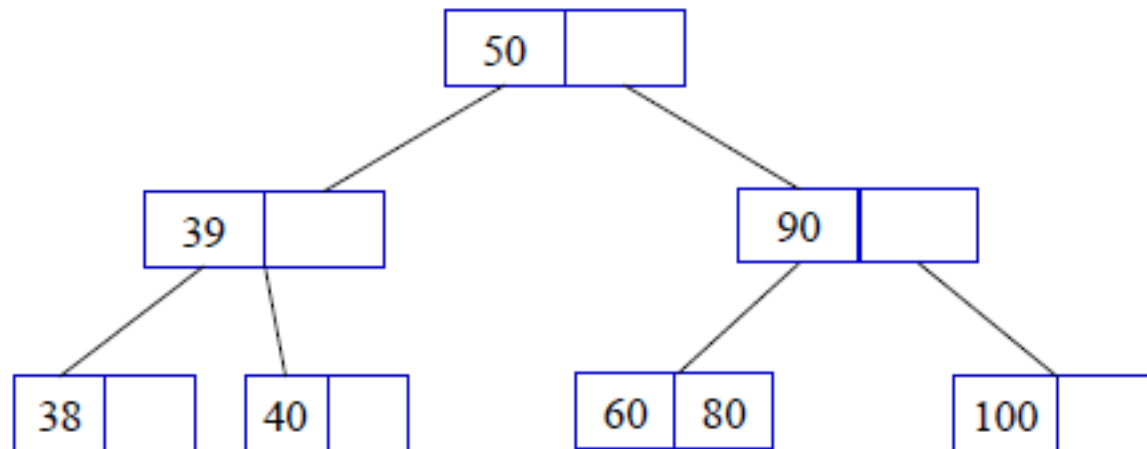
sigue →

- Se intenta escoger para fusionarse un hermano que tenga dos elementos,
 - para que el nodo padre no pierda elementos y evitar que se propaguen la fusiones hacia arriba.

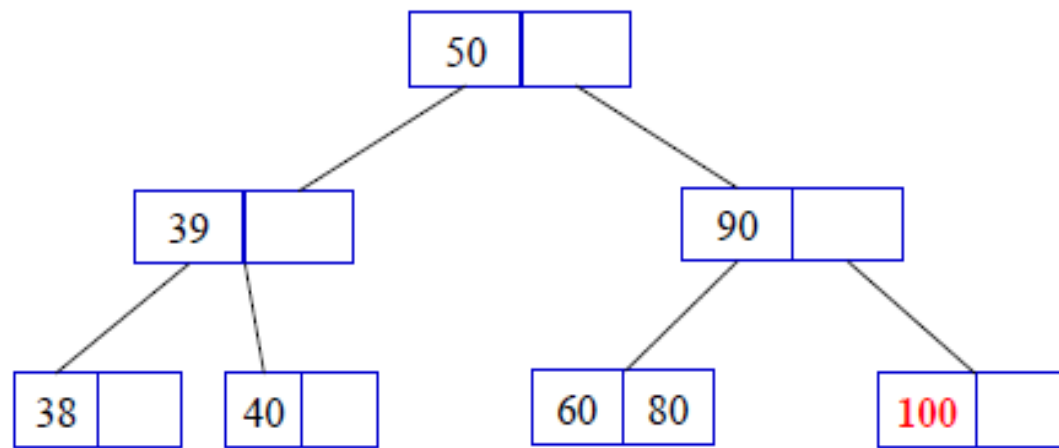


sigue →

- Como en este caso no hay ningún hermano que pueda ceder un elemento, se escoge un nodo cualquiera, por ejemplo el [60] y se baja un elemento del nodo padre el [80].
- El árbol queda finalmente:

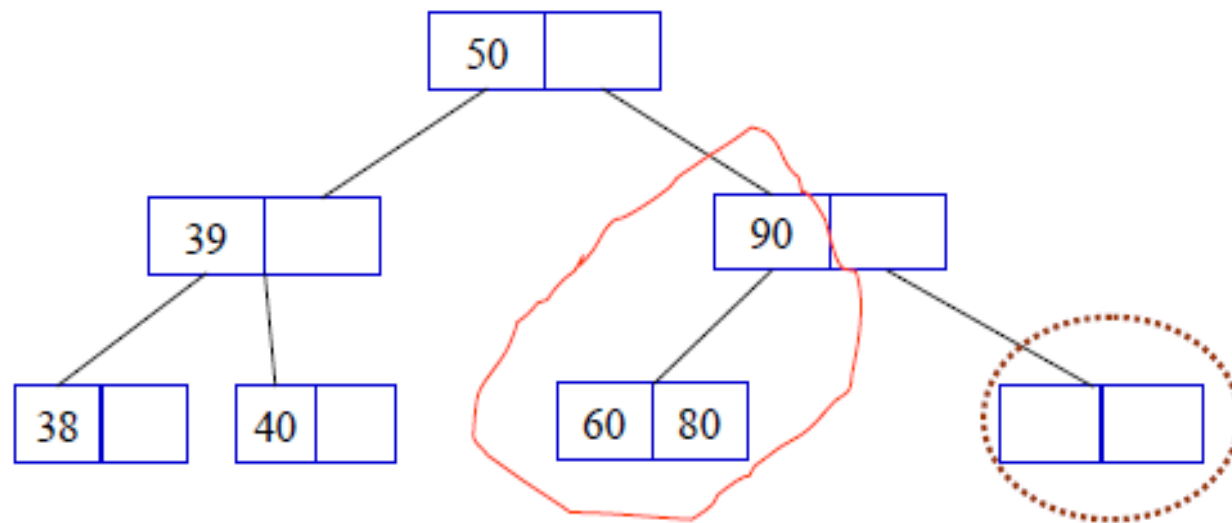


- Borrar el elemento **100**, que está en un nodo hoja.



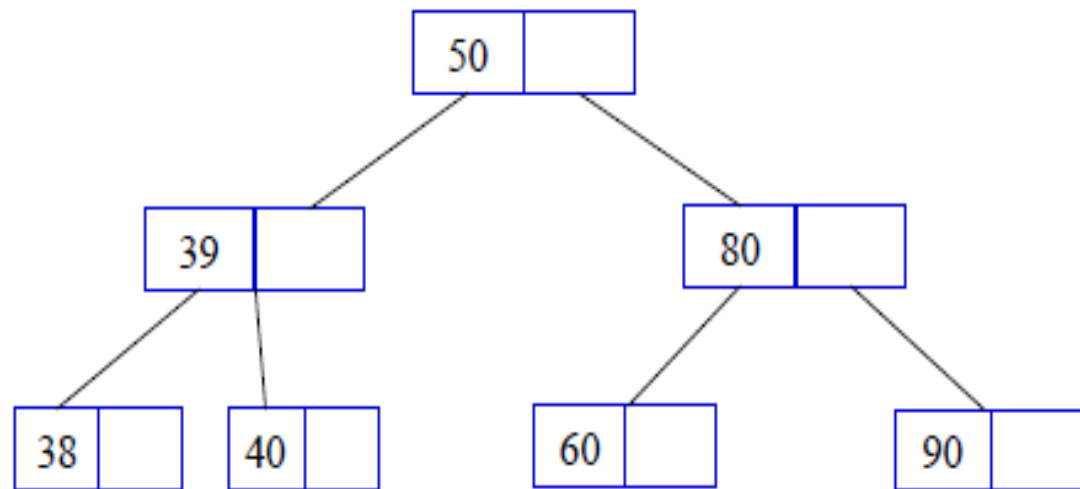
sigue →

- Este nodo se queda vacío, por lo que hay que realizar el proceso de fusión.



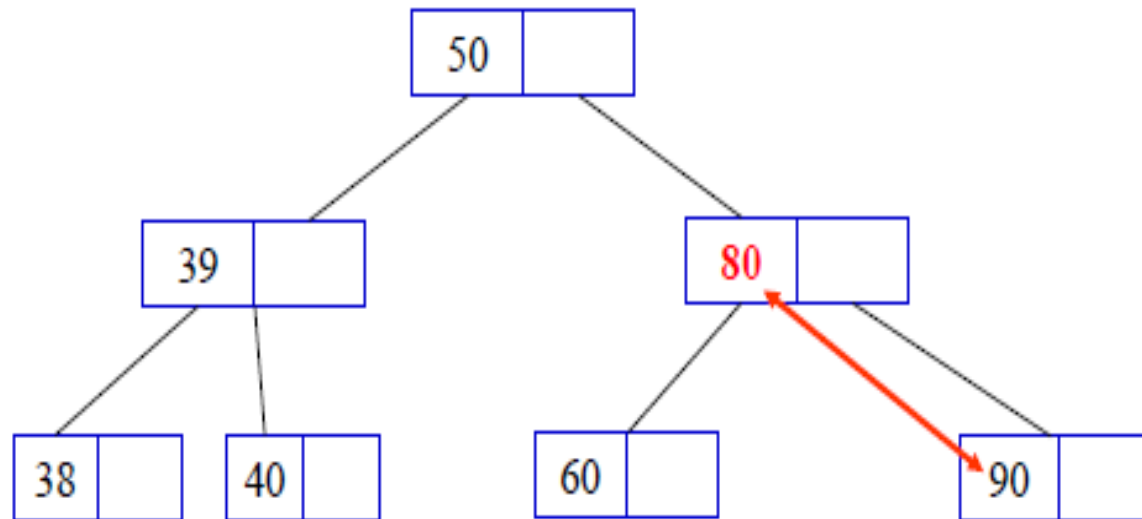
sigue →

- Como el nodo hermano está lleno, al bajar un elemento del padre, hay que dividirlo y repartir los elementos.
- Los nuevos nodos son el [60] y el [90] y el [80] es el elemento que pasa al nodo padre.



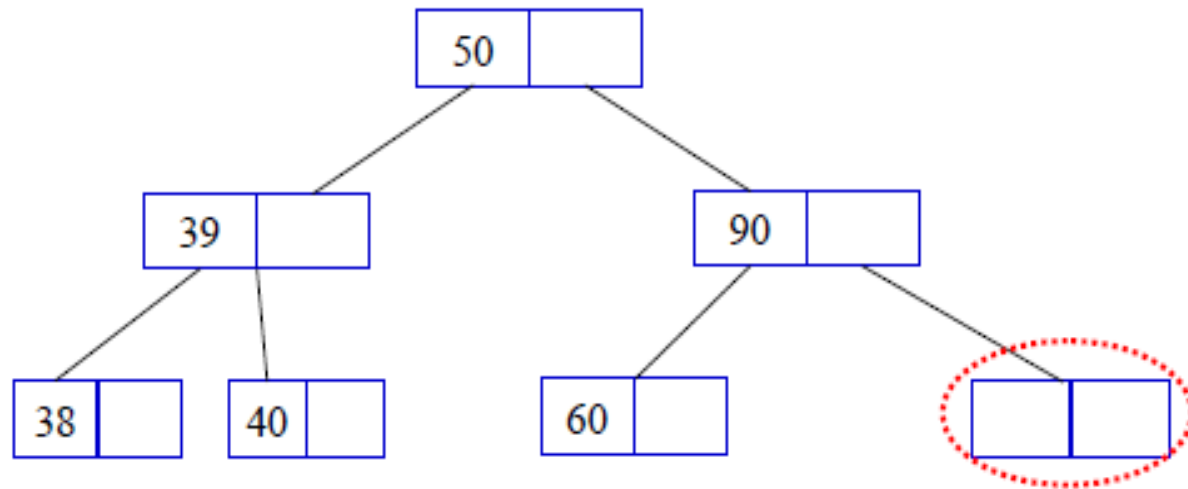
sigue →

- Borrar el elemento **80** que está en un nodo intermedio.
- El primer paso es intercambiarlo por su sucesor en inorden, el 90.



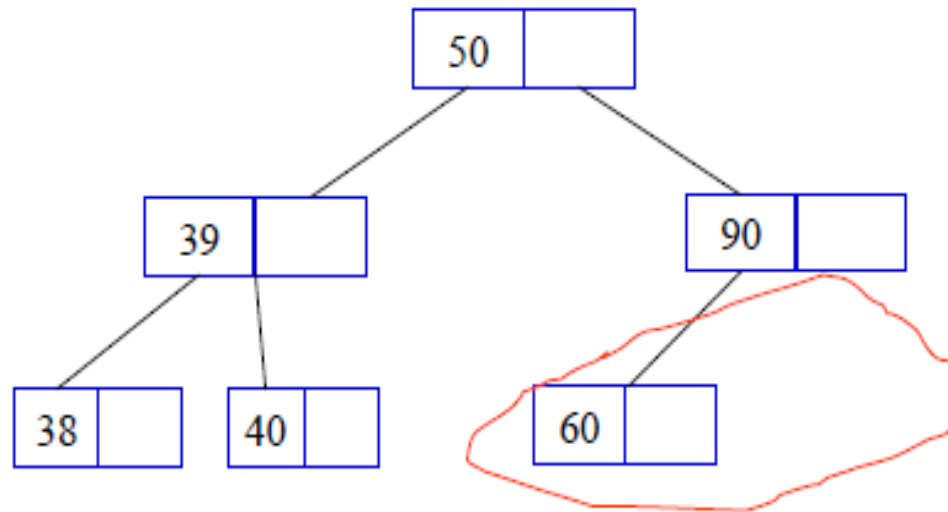
sigue →

- Resultado :



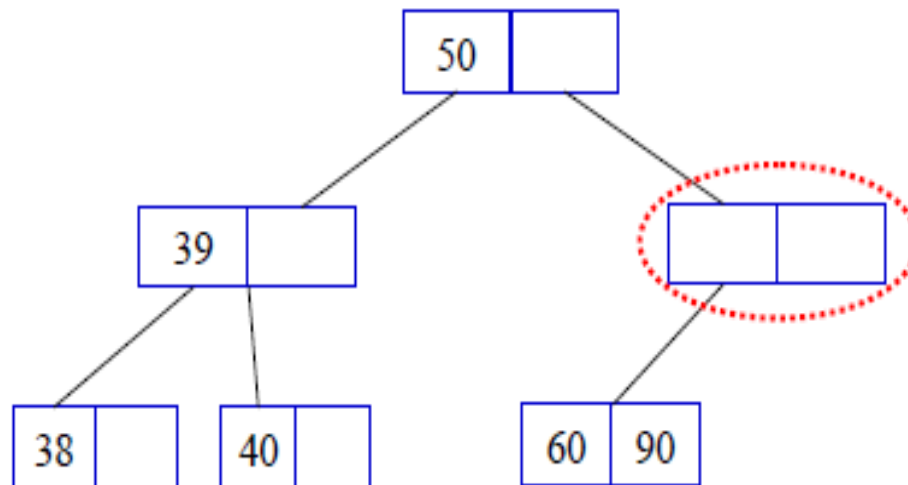
sigue →

- Al eliminar el 80 de la hoja donde se ha colocado, ésta se queda vacía, por lo que hay que fusionarla con su hermano.



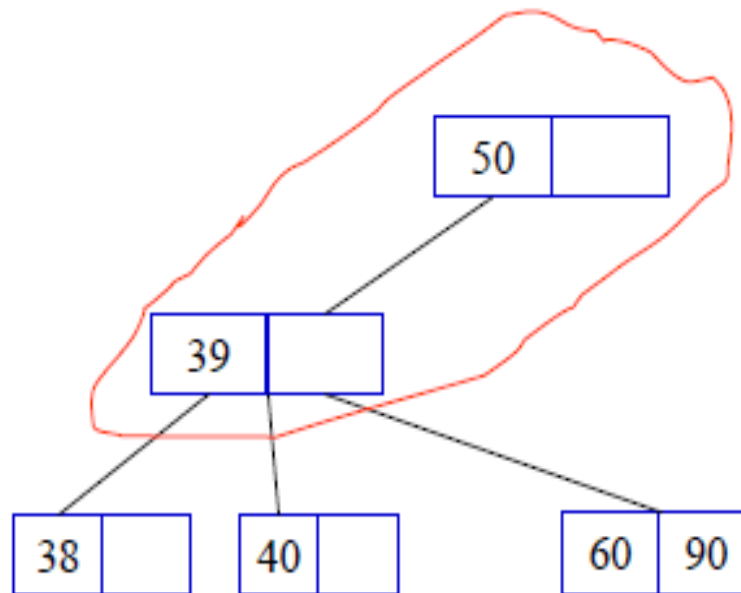
sigue →

- Su hermano, que no está lleno, acepta el elemento que baja del nodo padre, que se queda vacío :



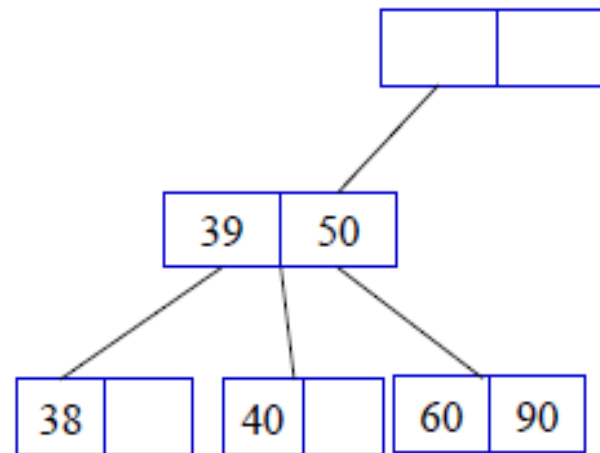
sigue →

- El nodo intermedio que se quedó vacío debe fusionarse con el hermano ([39]), que como no está lleno, acepta el elemento que baja de su padre, que se queda vacío.



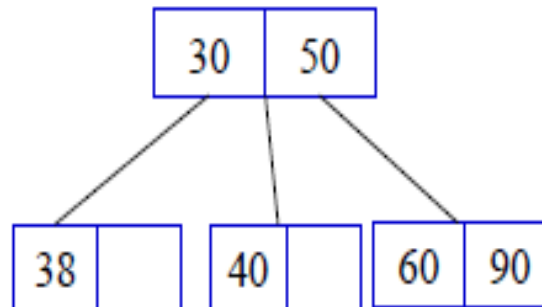
sigue →

- Como en el paso anterior, el hermano pasa a tener dos elementos y el padre, en este caso la raíz, se queda vacío:



sigue →

- Finalmente, se borra el nodo raíz vacío y su único hijo pasa a ser la nueva raíz.
- El árbol ha perdido altura:



Análisis de la complejidad

La altura h de un árbol 2-3 en el peor de los casos es cuando un árbol binario completo:

- $N = 1 + 2 + 4 + \dots + 2^{h-1} = (2^h - 1)(2 - 1) = 2^h - 1$ nodos, es decir $h \leq \log_2(n + 1)$
- En el mejor de los casos un árbol 2-3 sería:
- $N = 1 + 3 + 9 + \dots + 3^{h-1} = (3^h - 1)/(3 - 1) = (3^h - 1)/2$ nodos, es decir $h \geq \log_3(2n + 1)$

Luego la altura está entre $\log_2(n)$ y $\log_3(n)$